

```

// -----
// 1. main.cpp ** without assistance **
// -----
#include <iostream>
#include "menu.h"
#include "weatherManager.h"
int main() {
    int choice;

    while (true) {
        showMenu();
        std::cout << "Enter your choice: ";
        std::cin >> choice;
        if (choice == 7) break; // Exit on choice 7
        handleMenuChoice(choice);
    }

    return 0;
}
// -----
// 2. menu.h ** without assistance **
// -----
#ifndef MENU_H
#define MENU_H

void showMenu(); // Function to display the menu
void handleMenuChoice(int choice); // Function to handle the user's
choice

#endif
// -----
// 3. menu.cpp ** without assistance - all except case 4 **
// -----
#include "menu.h"
#include <iostream>
#include "weatherManager.h"
#include "candlestickData.h"
#include "plotData.h"
#include "filterData.h"
#include "predictData.h"

bool dataLoaded = false; // Variable to track if data has been
loaded
weatherManager manager; // Global instance of weatherManager

void showMenu() {
    std::cout << "=====\n";
    std::cout << "        Weather Analysis Toolkit\n";
    std::cout << "=====\n";
    std::cout << "Please choose an option from the menu below:\n";
    std::cout << "1. Load Data - Please load data to proceed with
below options\n";
    std::cout << "2. Compute Candlestick Data\n";
    std::cout << "3. Create Text-Based Candlestick Plot\n";

```

```

        std::cout << "4. Filter Data\n";
        std::cout << "5. Predict Weather Data\n";
        std::cout << "6. Help\n";
        std::cout << "7. Exit\n";
    }

void showHelp() {
    std::cout << "Help Menu:\n";
    std::cout << "1. Load Data: Load weather data from a file.\n";
    std::cout << "2. Compute Candlestick Data: Generate Yearly
Candlestick Data (Open|Close|High|Low) - Global or Country-Specific.
\n";
    std::cout << "3. Create Text-Based Candlestick Plot: 5-year
range.\n";
    std::cout << "4. Filter Data: Filter the loaded data based on
criteria - Country, Date Range or Averages.\n";
    std::cout << "5. Predict Weather Data: Predict future
temperature changes.\n";
    std::cout << "6. Help: Show this help menu.\n";
    std::cout << "7. Exit: Close the application.\n";
}

void handleMenuChoice(int choice) {
    switch (choice) {
        case 1:
            if (dataLoaded) {
                std::cout << "Data has already been loaded. Would
you like to reload from a different file? (y/n): ";
                char reloadChoice;
                std::cin >> reloadChoice;
                if (reloadChoice == 'y' || reloadChoice == 'Y') {
                    manager.clearData(); // Clear the current data
                    dataLoaded = false; // Reset the dataLoaded
flag
                    std::cout << "Data cleared. Please enter the
path to the new weather data file: ";
                } else {
                    std::cout << "Returning to menu...\n";
                    break;
                }
            }

            if (!dataLoaded) {
                std::string filePath;
                std::cout << "Enter the path to the weather data
file: ";
                std::cin >> filePath;

                manager.loadData(filePath); // Use weatherManager to
load data
                dataLoaded = true;

                // Show the first 5 rows of data after loading
                manager.showPreview();
            }
        }
    }
}

```

```

    }
    break;

case 2: {
    std::cout << "Would you like to see data for:\n";
    std::cout << "1. Global (all countries)\n";
    std::cout << "2. A specific country\n";
    std::cout << "Enter your choice (1 or 2): ";
    int subChoice;
    std::cin >> subChoice;

    std::string selectedCountryCode = "global";

    if (subChoice == 2) {
        std::cout << "Available country codes from the data
file:\n";
        std::set<std::string> countryCodes =
manager.getCountryCodes();
        for (const auto& code : countryCodes) {
            std::cout << code << "\n";
        }

        std::cout << "Enter the country code to analyze: ";
        std::cin >> selectedCountryCode;
    }

    manager.computeCandlestickData(selectedCountryCode);

    std::cout << "Would you like to view the computed
candlestick data now? (y/n): ";
    char viewChoice;
    std::cin >> viewChoice;

    if (viewChoice == 'y' || viewChoice == 'Y') {
        const auto& candlesticks =
manager.getCandlestickData();
        if (candlesticks.empty()) {
            std::cout << "No candlestick data available.\n";
        } else {
            std::cout << "----- Candlestick Details -----
\n";
            for (const auto& c : candlesticks) {
                c.print();
            }
        }
    } else {
        std::cout << "Returning to main menu without
displaying candlestick data...\n";
    }
    break;
}

case 3:
    plotData();

```

```

        break;

    case 4: {
        std::cout << "Choose a filter type:\n";
        std::cout << "1. Filter by Country\n";
        std::cout << "2. Filter by Date Range\n";
        std::cout << "3. Compute Averages\n";
        std::cout << "Enter your choice: ";
        int filterChoice;
        std::cin >> filterChoice;

        if (filterChoice == 1) {
            std::cout << "Available country codes from the data
file:\n";

            std::set<std::string> countryCodes =
manager.getCountryCodes();
            for (const auto& code : countryCodes) {
                std::cout << code << "\n";
            }

            std::string selectedCountryCode;
            std::cout << "Enter the country code to filter by:
";

            std::cin >> selectedCountryCode;

            const auto& filteredData =
manager.getRawWeatherDataByCountry(selectedCountryCode);
            if (filteredData.empty()) {
                std::cout << "No data available for the selected
country.\n";
            } else {
                std::cout << "Filtered Data for " <<
selectedCountryCode << ":\n";
                for (const auto& row : filteredData) {
                    std::cout << "Timestamp: " << row.timestamp
<< ", Temperature: " << row.temperatures.front() << "\n";
                }
            }
        } else if (filterChoice == 2) {
            std::string startDate, endDate;
            std::cout << "Enter the start date (YYYY-MM-DD): ";
            std::cin >> startDate;
            std::cout << "Enter the end date (YYYY-MM-DD): ";
            std::cin >> endDate;

            const auto& filteredData =
manager.getRawWeatherDataByDateRange(startDate, endDate);
            if (filteredData.empty()) {
                std::cout << "No data available for the
specified date range.\n";
            } else {
                std::cout << "Filtered Weather Data:\n";
                for (const auto& row : filteredData) {
                    std::cout << "Timestamp: " << row.timestamp

```

```

<< ", Temperatures: ";
        for (double temp : row.temperatures) {
            std::cout << temp << " ";
        }
        std::cout << "\n";
    }
} else if (filterChoice == 3) {
    std::cout << "Would you like to compute averages
for:\n";

    std::cout << "1. Global data\n";
    std::cout << "2. A specific country\n";
    std::cout << "Enter your choice (1 or 2): ";
    int subChoice;
    std::cin >> subChoice;

    std::string selectedCountryCode = "global";

    if (subChoice == 2) {
        std::cout << "Available country codes:\n";
        std::set<std::string> countryCodes =
manager.getCountryCodes();
        for (const auto& code : countryCodes) {
            std::cout << code << "\n";
        }

        std::cout << "Enter the country code to analyze:
";

        std::cin >> selectedCountryCode;
    }

    std::cout << "Choose the granularity for averages:
\n";

    std::cout << "1. Daily\n";
    std::cout << "2. Monthly\n";
    std::cout << "3. Yearly\n";
    std::cout << "Enter your choice: ";
    int granularityChoice;
    std::cin >> granularityChoice;

    std::string granularity;
    if (granularityChoice == 1) granularity = "daily";
    else if (granularityChoice == 2) granularity =
"monthly";
    else if (granularityChoice == 3) granularity =
"yearly";
    else {
        std::cout << "Invalid granularity choice.
Returning to menu...\n";
        break;
    }

    std::string startDate, endDate;
    std::cout << "Enter the start date (YYYY-MM-DD): ";

```

```

        std::cin >> startDate;
        std::cout << "Enter the end date (YYYY-MM-DD): ";
        std::cin >> endDate;

        auto averages =
manager.computeAverages(selectedCountryCode, startDate, endDate,
granularity);
        if (averages.empty()) {
            std::cout << "No averages computed for the
selected range and granularity.\n";
        } else {
            std::cout << "Averages computed successfully.
Plotting...\n";
            plotAveragesAsBarGraph(averages);
        }
    } else {
        std::cout << "Invalid choice. Returning to main
menu...\n";
    }
    break;
}

case 5: {
    std::cout << "Available country codes for prediction:
\n";
    std::set<std::string> countryCodes =
manager.getCountryCodes();
    for (const auto& code : countryCodes) {
        std::cout << code << "\n";
    }

    std::string countryCode;
    int startMonth, endMonth, predictionYear;

    std::cout << "Enter the country code for prediction
(e.g., AT_temperature): ";
    std::cin >> countryCode;
    std::cout << "Enter the start month for prediction
(1-12): ";
    std::cin >> startMonth;
    std::cout << "Enter the end month for prediction (1-12):
";
    std::cin >> endMonth;
    std::cout << "Enter the prediction year (e.g., 2022): ";
    std::cin >> predictionYear;

    double tempDifference =
predictTemperatureDifference(countryCode, startMonth, endMonth,
predictionYear);
    std::cout << "Predicted temperature difference between "
        << startMonth << " and " << endMonth
        << " in " << predictionYear << ": " <<
tempDifference << "°C\n";
}
}

```

```

        // Assuming the predicted temperatures for the months
        are returned by the prediction function
        std::vector<double> predictedTemps =
        getPredictedTemperatures(countryCode, startMonth, endMonth,
        predictionYear);

        // Plot the bar graph with the predicted temperatures
        plotTemperatureTimeSeries(startMonth, endMonth,
        predictedTemps);
        break;
    }

    case 6:
        showHelp();
        break;

    case 7:
        std::cout << "Exiting the application...\n";
        break;

    default:
        std::cout << "Invalid choice. Please try again.\n";
        break;
}
}
// -----
// 4. weatherManager.h ** without assistance **
// -----
#ifndef WEATHERMANAGER_H
#define WEATHERMANAGER_H

#include <vector>
#include <string>
#include <set>
#include <map>
#include "candlestickData.h"

struct WeatherData {
    std::string timestamp;           // e.g. "1980-01-01 01:00"
    std::vector<double> temperatures;
};

// weatherManger class
class weatherManager {
private:
    std::vector<WeatherData> weatherData;    // Raw data loaded from
the CSV
    std::vector<Candlestick> candlestickData; // Computed yearly
candlestick data
    std::string dataFilePath;               // File path for the
loaded data

public:
    // Loads the CSV file and stores it in weatherData

```

```

void loadData(const std::string& filePath);

// Returns a reference to the raw loaded data
const std::vector<WeatherData>& getWeatherData() const;

// Displays a preview (first 'n' rows) of the raw loaded data
void showPreview(size_t rows = 5) const;

// Clears all loaded data and candlestick data
void clearData() {
    weatherData.clear();
    candlestickData.clear();
    dataFilePath.clear();
}

// Computes candlestick data aggregated by year, with optional
country filtering
void computeCandlestickData(const std::string&
selectedCountryCode = "global");

// Retrieves the available country codes from the data
std::set<std::string> getCountryCodes() const;

// Filters RAW weather data by a specific country code
std::vector<WeatherData> getRawWeatherDataByCountry(const
std::string& countryCode) const;

// Filters RAW weather data by a specific date range
std::vector<WeatherData> getRawWeatherDataByDateRange(const
std::string& startDate, const std::string& endDate) const;

// Access the computed candlestick data
const std::vector<Candlestick>& getCandlestickData() const;

// FILTERED candlestick data by a specific country code
std::vector<Candlestick>
getFilteredCandlestickDataByCountry(const std::string& countryCode)
const;

// FILTERED candlestick data for a specific date range
std::vector<Candlestick>
getFilteredCandlestickDataByDateRange(int startYear, int endYear)
const;

// Helper function for computing averages
std::map<std::string, double> computeAverages(
    const std::string& countryCode, const std::string&
startDate, const std::string& endDate, const std::string&
granularity) const;

// Computes daily averages for a given date range
std::map<std::string, double> computeDailyAverages(
    const std::string& countryCode, const std::string&
startDate, const std::string& endDate) const;

```



```

        // Computes monthly averages for a given date range
        std::map<std::string, double> computeMonthlyAverages(
            const std::string& countryCode, const std::string&
startDate, const std::string& endDate) const;

        // Computes yearly averages for a given date range
        std::map<std::string, double> computeYearlyAverages(
            const std::string& countryCode, const std::string&
startDate, const std::string& endDate) const;
};

#endif // WEATHERMANAGER_H
// -----
// 5. weatherManager.cpp
// -----
#include "weatherManager.h"
#include <fstream>
#include <sstream>
#include <map>
#include <set>
#include <algorithm>
#include <numeric>
#include <iostream>

// To store column headers for identifying country codes
std::vector<std::string> headers;

// -----
// 1) Load raw weather data from CSV
// -----
void weatherManager::loadData(const std::string& filePath) {
    dataFilePath = filePath; // Save the file path
    std::ifstream file(filePath);
    if (!file) {
        std::cerr << "Error: Could not open file " << filePath <<
std::endl;
        return;
    }

    std::string line;
    bool isFirstLine = true;

    while (std::getline(file, line)) {
        std::stringstream ss(line);
        std::string value;

        if (isFirstLine) {
            // Extract headers from the first line
            std::getline(ss, value, ','); // Skip "utc_timestamp"
            while (std::getline(ss, value, ',')) {
                headers.push_back(value);
            }
            isFirstLine = false;

```

```

        continue;
    }

    WeatherData row;
    std::getline(ss, row.timestamp, ','); // First column is the
timestamp

    // Remaining columns are temperatures for different
countries
    while (std::getline(ss, value, ',')) {
        try {
            row.temperatures.push_back(std::stod(value));
        } catch (const std::invalid_argument& e) {
            std::cerr << "Error: Invalid temperature value in
file. Setting to 0.\n";
            row.temperatures.push_back(0.0); // Handle invalid
data gracefully
        }
    }

    weatherData.push_back(row);
}

std::cout << "Data loaded successfully from " << filePath
    << ". Total rows: " << weatherData.size() <<
std::endl;
}

// -----
// 2) Access the loaded weather data
// -----
const std::vector<WeatherData>& weatherManager::getWeatherData()
const {
    return weatherData;
}

// -----
// 3) Show a preview of the first 'n' rows of raw data ** without
assistance **
// -----
void weatherManager::showPreview(size_t rows) const {
    size_t count = 0;
    for (const auto& data : weatherData) {
        std::cout << "Timestamp Preview: " << data.timestamp << ",
Temperatures: ";
        for (double temp : data.temperatures) {
            std::cout << temp << " ";
        }
        std::cout << std::endl;

        if (++count >= rows) break;
    }
}

```

```

// -----
// 4) Get Unique Country Codes from Headers ** without assistance **
// -----
std::set<std::string> weatherManager::getCountryCodes() const {
    std::set<std::string> countryCodes;

    std::ifstream file(dataFilePath);
    if (!file.is_open()) {
        std::cerr << "Error: Unable to open the file for extracting
headers." << std::endl;
        return countryCodes;
    }

    std::string line;
    if (std::getline(file, line)) {
        std::stringstream ss(line);
        std::string header;

        // Skip the first column header ("utc_timestamp")
        std::getline(ss, header, ',');

        // Collect remaining headers as country codes
        while (std::getline(ss, header, ',')) {
            countryCodes.insert(header);
        }
    }

    return countryCodes;
}

// -----
// 5) Filter Raw Weather Data by Country
// -----
std::vector<WeatherData>
weatherManager::getRawWeatherDataByCountry(const std::string&
countryCode) const {
    std::vector<WeatherData> filteredData;

    // Check if the country code exists in the headers
    auto it = std::find(headers.begin(), headers.end(),
countryCode);
    if (it == headers.end()) {
        std::cerr << "Error: Invalid country code '" << countryCode
<< "' provided.\n";
        return filteredData;
    }

    size_t countryIndex = std::distance(headers.begin(), it);

    // Filter data for the selected country
    for (const auto& data : weatherData) {
        if (countryIndex < data.temperatures.size()) {
            WeatherData filteredRow = data;
            filteredRow.temperatures =

```

```

    { data.temperatures[countryIndex] }; // Keep only the temperature
    for the selected country
        filteredData.push_back(filteredRow);
    }
}

return filteredData;
}

// -----
// 6) Filter Raw Weather Data by Date Range
// -----
std::vector<WeatherData>
weatherManager::getRawWeatherDataByDateRange(const std::string&
startDate, const std::string& endDate) const {
    std::vector<WeatherData> filteredData;

    // Iterate over the weather data
    for (const auto& data : weatherData) {
        if (data.timestamp >= startDate && data.timestamp <=
endDate) {
            filteredData.push_back(data); // Add rows within the
date range
        }
    }

    return filteredData;
}

// -----
// 7) Compute Candlestick Data
// -----
void weatherManager::computeCandlestickData(const std::string&
selectedCountryCode) {
    candlestickData.clear();
    std::map<int, std::vector<double>> yearToTemps;

    // Find the index of the selected country code
    auto it = std::find(headers.begin(), headers.end(),
selectedCountryCode);
    if (it == headers.end() && selectedCountryCode != "global") {
        std::cerr << "Error: Invalid country code '" <<
selectedCountryCode << "' provided.\n";
        return;
    }

    size_t columnIndex = std::distance(headers.begin(), it);

    // Filter temperatures by year and country code
    for (const auto& row : weatherData) {
        if (selectedCountryCode == "global") {
            for (double temp : row.temperatures) {
                int year = std::stoi(row.timestamp.substr(0, 4));
                yearToTemps[year].push_back(temp);
            }
        }
    }
}

```

```

        }
    } else {
        if (columnIndex < row.temperatures.size()) {
            int year = std::stoi(row.timestamp.substr(0, 4));

yearToTemps[year].push_back(row.temperatures[columnIndex]);
        }
    }
}

// Compute candlestick data for each year
double prevClose = 0.0; // Default for the first year
bool isFirstYear = true;

for (const auto& [year, temps] : yearToTemps) {
    if (temps.empty()) continue;

    double high = *std::max_element(temps.begin(), temps.end());
    double low = *std::min_element(temps.begin(), temps.end());
    double close = std::accumulate(temps.begin(), temps.end(),
0.0) / temps.size();

    double open;
    if (isFirstYear) {
        open = temps.front(); // Use first temperature for the
first year
        isFirstYear = false;
    } else {
        open = prevClose; // Set Open to the previous year's
Close
    }

    // Update prevClose for the next iteration
    prevClose = close;

    candlestickData.emplace_back(std::to_string(year), open,
high, low, close);
}

    std::cout << "Candlestick data computed for "
        << (selectedCountryCode == "global" ? "global" :
selectedCountryCode)
        << ". Total candlesticks: " << candlestickData.size()
<< "\n";
}

// -----
// 8) Filter Candlestick Data by Country
// -----
std::vector<Candlestick>
weatherManager::getFilteredCandlestickDataByCountry(const
std::string& countryCode) const {
    std::vector<Candlestick> filteredCandlesticks;

```

```

        // Check if the country code exists in the headers
        auto it = std::find(headers.begin(), headers.end(),
countryCode);
        if (it == headers.end()) {
            std::cerr << "Error: Invalid country code '" << countryCode
<< "' provided.\n";
            return filteredCandlesticks;
        }

        size_t countryIndex = std::distance(headers.begin(), it);

        // Filter candlesticks for the selected country
        for (const auto& candle : candlestickData) {
            if (countryCode == "global" || headers[countryIndex] ==
countryCode) {
                filteredCandlesticks.push_back(candle);
            }
        }

        return filteredCandlesticks;
    }

// -----
// 9) Filter Candlestick Data by Date Range
// -----
std::vector<Candlestick>
weatherManager::getFilteredCandlestickDataByDateRange(int startYear,
int endYear) const {
    std::vector<Candlestick> filteredData;
    for (const auto& candle : candlestickData) {
        int year = std::stoi(candle.getDate());
        if (year >= startYear && year <= endYear) {
            filteredData.push_back(candle);
        }
    }
    return filteredData;
}

// -----
// 10) Get Computed Candlestick Data
// -----
const std::vector<Candlestick>& weatherManager::getCandlestickData()
const {
    return candlestickData;
}

// -----
// 11) Helper for Averages
// -----
std::string extractDatePart(const std::string& timestamp, const
std::string& granularity) {
    if (granularity == "day" || granularity == "daily") {
        if (timestamp.size() >= 10) return timestamp.substr(0,
10); // YYYY-MM-DD
    }
}

```

```

    }
    if (granularity == "month" || granularity == "monthly") {
        if (timestamp.size() >= 7) return timestamp.substr(0, 7); //
YYYY-MM
    }
    if (granularity == "year" || granularity == "yearly") {
        if (timestamp.size() >= 4) return timestamp.substr(0, 4); //
YYYY
    }
    std::cerr << "Error: Invalid timestamp or granularity: " <<
timestamp << ", " << granularity << "\n";
    return "";
}

// Generic function to compute averages
std::map<std::string, double> weatherManager::computeAverages(
    const std::string& countryCode, const std::string& startDate,
const std::string& endDate, const std::string& granularity) const {
    std::map<std::string, std::vector<double>> groupedTemps;
    size_t countryIndex = 0;

    if (countryCode != "global") {
        auto it = std::find(headers.begin(), headers.end(),
countryCode);
        if (it == headers.end()) {
            std::cerr << "Error: Invalid country code '" <<
countryCode << "' provided.\n";
            return {};
        }
        countryIndex = std::distance(headers.begin(), it);
    }

    std::cout << "Processing weather data for country code '" <<
countryCode
        << "' between '" << startDate << "' and '" << endDate
<< "'\n";

    for (const auto& row : weatherData) {
        if (row.timestamp < startDate || row.timestamp > endDate)
continue;

        std::string key = extractDatePart(row.timestamp,
granularity);
        if (key.empty()) continue;

        if (countryCode == "global") {
            groupedTemps[key].insert(groupedTemps[key].end(),
row.temperatures.begin(), row.temperatures.end());
        } else if (countryIndex < row.temperatures.size()) {
groupedTemps[key].push_back(row.temperatures[countryIndex]);
        }
    }
}

```

```

        std::map<std::string, double> averages;
        for (const auto& [key, temps] : groupedTemps) {
            if (!temps.empty()) {
                averages[key] = std::accumulate(temps.begin(),
temps.end(), 0.0) / temps.size();
                std::cout << "Average for " << key << " = " <<
averages[key] << "\n";
            }
        }

        if (averages.empty()) {
            std::cerr << "No averages computed for the selected range
and granularity.\n";
        }

        return averages;
    }

// Daily averages
std::map<std::string, double> weatherManager::computeDailyAverages(
    const std::string& countryCode, const std::string& startDate,
const std::string& endDate) const {
    return computeAverages(countryCode, startDate, endDate, "day");
}

// Monthly averages
std::map<std::string, double>
weatherManager::computeMonthlyAverages(
    const std::string& countryCode, const std::string& startDate,
const std::string& endDate) const {
    return computeAverages(countryCode, startDate, endDate,
"month");
}

// Yearly averages
std::map<std::string, double> weatherManager::computeYearlyAverages(
    const std::string& countryCode, const std::string& startDate,
const std::string& endDate) const {
    return computeAverages(countryCode, startDate, endDate, "year");
}

// -----
// 6. candlestickData.h ** without assistance **
// -----
#ifndef CANDLESTICKDATA_H
#define CANDLESTICKDATA_H

#include <string>
#include <iostream>

// Candlestick Class
class Candlestick {
private:
    std::string date; // could be "YYYY", "YYYY-MM", or full "YYYY-
MM-DD

```



```

        double open; // average mean temperature of the previous time
frame
        double high; // highest temperature in the current time frame
        double low; // lowest temperature in the current time frame
        double close; // average mean temperature of the current time
frame

public:
    // Constructor to initialize candlestick data
    Candlestick(const std::string& date, double open, double high,
double low, double close)
        : date(date), open(open), high(high), low(low), close(close)
    {}

    // Getters
    std::string getDate() const { return date; }
    double      getOpen() const { return open; }
    double      getHigh() const { return high; }
    double      getLow()  const { return low; }
    double      getClose() const { return close; }

    // For debugging or text-based display
    void print() const {
        std::cout << "Date: " << date
                    << ", Open: " << open
                    << ", High: " << high
                    << ", Low: " << low
                    << ", Close: " << close << std::endl;
    }
};

#endif // CANDLESTICKDATA_H
// -----
// 7. candlestickData.cpp
// -----
#include "candlestickData.h"
#include "weatherManager.h"
#include <iostream>
#include <vector>
#include <numeric>
#include <algorithm>
#include <map>
#include <string>

// Helper function to extract the year from a timestamp (assuming
format "YYYY-MM-DD HH:MM:SS")
std::string extractYear(const std::string& timestamp) {
    return timestamp.substr(0, 4); // First 4 characters represent
the year
}

extern weatherManager manager; // Declare manager as extern

void computeCandlestickData() {

```

```

std::cout << "Computing candlestick data...\n";

// Get weather data from manager
std::vector<WeatherData> weatherData = manager.getWeatherData();

std::map<std::string, std::vector<double>> yearlyData; // Map to
group temperatures by year

// Group temperatures by year
for (const auto& data : weatherData) {
    std::string year = extractYear(data.timestamp);
    yearlyData[year].insert(yearlyData[year].end(),
data.temperatures.begin(), data.temperatures.end());
}

std::vector<Candlestick> candlestickData;

// Compute candlestick data for each year
std::string prevYear = ""; // To track the previous year
double prevClose = 0.0;    // To compute Open for the current
year

for (const auto& [year, temperatures] : yearlyData) {
    double open = (prevYear.empty()) ? 0.0 : prevClose; // Use
previous year's Close as Open
    double close = std::accumulate(temperatures.begin(),
temperatures.end(), 0.0) / temperatures.size(); // Average of
current year's temperatures
    double high = *std::max_element(temperatures.begin(),
temperatures.end()); // Highest temperature
    double low = *std::min_element(temperatures.begin(),
temperatures.end()); // Lowest temperature

    // Update prevYear and prevClose for the next iteration
    prevYear = year;
    prevClose = close;

    // Add the candlestick data
    candlestickData.push_back(Candlestick{year, open, high, low,
close});
}

// Display the candlestick data
std::cout << "Candlestick Data:\n";
std::cout << "Date      Open   High   Low    Close\n";
std::cout << "=====\n";
for (const auto& candlestick : candlestickData) {
    candlestick.print();
}
}
// -----
// 8. plotData.h ** without assistance **
// -----
#endif PLOTDATA_H

```

```

#define PLOTDATA_H

#include <vector>          // For std::vector
#include "weatherManager.h" // For WeatherData type

// Function to plot the data
void plotData();

void plotAveragesAsBarGraph(const std::map<std::string, double>&
averages);

void plotTemperatureTimeSeries(int startMonth, int endMonth, const
std::vector<double>& predictedTemps);

#endif
// -----
// 9. plotData.cpp
// -----
#include "plotData.h"
#include "weatherManager.h"
#include <iostream>
#include <iomanip>
#include <vector>
#include <string>
#include <algorithm>

extern weatherManager manager; // Access the global weatherManager
instance

// Helper function: Normalize a temperature value to a grid row
// (0-8)
int normalizeTemperature(double temp, double minTemp, double
maxTemp, int rows) {
    return static_cast<int>((temp - minTemp) / (maxTemp - minTemp) *
rows);
}

void printCandlestickData(const std::vector<Candlestick>&
filteredData) {
    // Print the candlestick data for the selected 5-year range
    std::cout << "\nCandlestick Data for Selected Range:\n";
    std::cout << "Year    Open    High    Low    Close\n";
    std::cout << "=====\n";

    for (const auto& candle : filteredData) {
        std::cout << candle.getDate() << " | "
            << std::setw(6) << candle.getOpen() << " | "
            << std::setw(6) << candle.getHigh() << " | "
            << std::setw(6) << candle.getLow() << " | "
            << std::setw(6) << candle.getClose() << "\n";
    }
}

// Function to print the ASCII candlestick graph

```

```

void printCandlestickGraph(const std::vector<Candlestick>&
filteredData) {
    const double minTemp = -40.0, maxTemp = 40.0;
    const int rows = 20; // Number of rows for graphing
    const int candlestickPadding = 5; // Padding between
candlesticks (adjust this value as needed)

    // Print the graph header
    std::cout << "\nCandlestick Graph for Selected Range:\n";
    std::cout << "Temperature (°C)\n";

    // Print the legend
    std::cout << "\nLegend: ";
    std::cout << "O = Open, C = Close, + = High, - = Low, | =
Candlestick Body\n";
    std::cout <<
"-----\n";

    // Loop through each row (y-axis)
    for (int row = rows; row >= 0; --row) {
        double temp = minTemp + row * (maxTemp - minTemp) / rows;
        std::cout << std::setw(3) << static_cast<int>(temp) << " |
"; // Print temperature on y-axis with 3 characters

        bool previousColumnHasCharacter = false; // Flag to track
previous column's character

        // Loop through each candlestick for the selected year range
        for (size_t i = 0; i < filteredData.size(); ++i) {
            // Shift the first candlestick to the right by 2 spaces
            if (i == 0) {
                std::cout << "  "; // Adding 2 spaces before the
first candlestick
            }

            int highRow =
normalizeTemperature(filteredData[i].getHigh(), minTemp, maxTemp,
rows);
            int lowRow =
normalizeTemperature(filteredData[i].getLow(), minTemp, maxTemp,
rows);
            int openRow =
normalizeTemperature(filteredData[i].getOpen(), minTemp, maxTemp,
rows);
            int closeRow =
normalizeTemperature(filteredData[i].getClose(), minTemp, maxTemp,
rows);

            int updatedCandlestickPadding = candlestickPadding;
            // Plot High to Low with vertical lines
            if (row <= highRow && row >= lowRow) {
                if (row == openRow && row == closeRow) {
                    std::cout << "OC"; // Both Open and Close on
the same row

```

```

        updatedCandlestickPadding -= 1;
    } else if (row == openRow) {
        std::cout << "O";
    } else if (row == closeRow) {
        std::cout << "C";
    } else if (row == highRow) {
        std::cout << "+";
    } else if (row == lowRow) {
        std::cout << "-";
    } else {
        std::cout << "|"; // Candlestick body
    }
} else {
    std::cout << " ";
}

// Add padding after each candlestick
for (int p = updatedCandlestickPadding; p > 0; --p) {
    std::cout << " ";
}

std::cout << "\n"; // New line after each row
}

// Print x-axis (Years)
std::cout << "      +";
for (size_t i = 0; i < filteredData.size(); ++i) {
    std::cout << "-----+"; // Adjust the width of the spaces
between each year
}
std::cout << "\n      "; // Adjust alignment to match the
candlesticks

// Loop through the filtered data and print the year labels with
additional spacing
for (size_t i = 0; i < filteredData.size(); ++i) {
    std::string year = filteredData[i].getDate().substr(2); //
Get last 2 digits of the year
    if (i == 0) {
        // Add extra space before the first year
        std::cout << " ";
    }
    std::cout << std::setw(4) << year << " "; // Print the
year with 4 spaces, plus 2 extra spaces - alignment issues (need to
fix)
}
std::cout << "\n";
}

void plotData() {
    // Get candlestick data from the manager
    const auto& candlesticks = manager.getCandlestickData();

    if (candlesticks.empty()) {

```

```

        std::cout << "No candlestick data available for plotting.
\n";
        return;
    }

    // Identify the available year ranges
    int startYear = std::stoi(candlesticks.front().getDate());
    int endYear = std::stoi(candlesticks.back().getDate());

    std::cout << "Available candlestick data ranges from " <<
startYear << " to " << endYear << ".\n";
    std::cout << "Enter the start year for the 5-year range you'd
like to view: ";
    int selectedStartYear;
    std::cin >> selectedStartYear;

    // Validate the selected range
    if (selectedStartYear < startYear || selectedStartYear > endYear
- 4) {
        std::cout << "Invalid range. Please select a valid start
year between " << startYear
        << " and " << endYear - 4 << ".\n";
        return;
    }

    int selectedEndYear = selectedStartYear + 4;

    std::cout << "You selected the range " << selectedStartYear << "
to " << selectedEndYear << ".\n";

    // Filter the candlestick data for the selected range
    std::vector<Candlestick> filteredData;
    for (const auto& candle : candlesticks) {
        int year = std::stoi(candle.getDate());
        if (year >= selectedStartYear && year <= selectedEndYear) {
            filteredData.push_back(candle);
        }
    }

    // Check if any data was found for the range
    if (filteredData.empty()) {
        std::cout << "No candlestick data available for the selected
range.\n";
        return;
    }

    // Print the candlestick data for the 5-year range
    printCandlestickData(filteredData);

    // Print the ASCII candlestick graph for the 5-year range
    printCandlestickGraph(filteredData);
}

```

```

void plotAveragesAsBarGraph(const std::map<std::string, double>&

```

```

averages) {
    if (averages.empty()) {
        std::cout << "No averages to plot.\n";
        return;
    }

    // Find the maximum average value for scaling
    double maxAverage = std::max_element(
        averages.begin(), averages.end(),
        [](const auto& a, const auto& b) { return a.second <
b.second; }
    )->second;

    const int maxBarLength = 50; // Maximum bar length in characters

    std::cout << "\nGraph (Bar Chart):\n";

    // Draw the bar graph
    for (const auto& [key, value] : averages) {
        int barLength = static_cast<int>((value / maxAverage) *
maxBarLength);

        std::cout << key << " | "; // Label (e.g., year/month)
        for (int i = 0; i < barLength; ++i) {
            std::cout << "#"; // Bar character
        }
        std::cout << " (" << std::fixed << std::setprecision(2) <<
value << ")\n"; // Show value
    }
    std::cout << std::endl;
}

void plotTemperatureTimeSeries(int startMonth, int endMonth, const
std::vector<double>& predictedTemps) {
    std::cout << "Predicted average temperatures (" << startMonth <<
"- " << endMonth << "):\n";

    // Temperature range for scaling
    const double minTemp = -10.0; // Min temperature (adjustable)
    const double maxTemp = 40.0; // Max temperature (adjustable)

    const int maxBarLength = 10; // Maximum number of '*' characters
for the bar

    double totalTemp = 0.0;
    int monthsCounted = endMonth - startMonth + 1;

    // Loop through the months and plot the temperatures
    for (int month = startMonth, i = 0; month <= endMonth; ++month,
++i) {
        double temp = predictedTemps[i];
        totalTemp += temp;

        // Calculate the number of stars for the bar based on the

```

```

temperature
    int numStars = static_cast<int>((temp - minTemp) / (maxTemp
- minTemp) * maxBarLength);

    // Output the month, stars, and actual temperature
    std::cout << std::setw(3) << month << " | ";
    for (int j = 0; j < numStars; ++j) {
        std::cout << "*";
    }
    std::cout << " (" << std::fixed << std::setprecision(1) <<
temp << "°C)\n";
}

// Calculate temperature difference between the first and last
month of the range
double tempDifference = predictedTemps[endMonth - startMonth] -
predictedTemps[0];

std::cout << "\nTemperature difference between month " <<
startMonth << " and month " << endMonth
        << " is: " << std::fixed << std::setprecision(1) <<
tempDifference << "°C\n";

// Print the average temperature for the selected range
double avgTemp = totalTemp / monthsCounted;
std::cout << "\nAverage temperature for the selected range (" <<
startMonth << "-" << endMonth
        << "): " << std::fixed << std::setprecision(1) <<
avgTemp << "°C\n";

std::cout << "Prediction complete.\n";
}
// -----
// 10. filterData.h ** without assistance **
// -----
#ifdef FILTERDATA_H
#define FILTERDATA_H

void filterData(); // Function to filter the data

#endif
// -----
// 11. filterData.cpp
// -----
#include "filterData.h"
#include "weatherManager.h"
#include <iostream>
#include <set>
#include <string>

extern weatherManager manager; // Access the global weatherManager
instance

void filterData() {

```



```

    std::cout << "Filtering data by country...\n";

    // Display available country codes
    std::set<std::string> countryCodes = manager.getCountryCodes();
    if (countryCodes.empty()) {
        std::cout << "No country codes available. Please ensure data
is loaded.\n";
        return;
    }

    std::cout << "Available country codes:\n";
    for (const auto& code : countryCodes) {
        std::cout << code << "\n";
    }

    // Ask the user to select a country code
    std::string selectedCountryCode;
    std::cout << "Enter the country code to filter by: ";
    std::cin >> selectedCountryCode;

    // Get filtered candlestick data for the selected country
    auto filteredData =
manager.getFilteredCandlestickDataByCountry(selectedCountryCode);

    if (filteredData.empty()) {
        std::cout << "No data available for the selected country: "
<< selectedCountryCode << "\n";
    } else {
        std::cout << "----- Filtered Candlestick Data for " <<
selectedCountryCode << " ----- \n";
        for (const auto& candlestick : filteredData) {
            candlestick.print();
        }
    }
}

// -----
// 12. predictData.h ** without assistance **
// -----
#ifndef PREDICTDATA_H
#define PREDICTDATA_H

#include <vector>
#include <string>

// Predict temperature difference between two months for a specific
year
double predictTemperatureDifference(
    const std::string& countryCode,
    int startMonth,
    int endMonth,
    int predictionYear);

// Function to get predicted temperatures for the specified months
and year

```

```

std::vector<double> getPredictedTemperatures(const std::string&
countryCode, int startMonth, int endMonth, int predictionYear);

#endif // PREDICTDATA_H
// -----
// 13. predictData.cpp
// -----
#include "predictData.h"
#include "weatherManager.h"
#include <iostream>
#include <vector>
#include <numeric>
#include <cmath>

extern weatherManager manager; // Access global weatherManager
instance

// Helper function to calculate historical monthly averages
std::vector<double> calculateHistoricalMonthlyAverages(const
std::string& countryCode, int month) {
    const auto& weatherData =
manager.getRawWeatherDataByCountry(countryCode);
    std::vector<double> monthlyAverages;

    // Loop through historical data
    for (int year = 1980; year <= 2019; ++year) {
        std::vector<double> temperatures;

        // Extract temperatures for the specified month and year
        for (const auto& data : weatherData) {
            if (std::stoi(data.timestamp.substr(0, 4)) == year &&
                std::stoi(data.timestamp.substr(5, 2)) == month) {
                temperatures.push_back(data.temperatures.front());
            }
        }

        if (!temperatures.empty()) {
            double average = std::accumulate(temperatures.begin(),
temperatures.end(), 0.0) / temperatures.size();
            monthlyAverages.push_back(average);
        }
    }

    return monthlyAverages;
}

// Linear regression prediction for a single month
double predictMonthlyTemperature(const std::vector<double>&
historicalAverages, int predictionYear) {
    int startYear = 1980;
    int endYear = 2019;

    // Prepare x (years) and y (average temperatures)
    std::vector<int> years;

```

```

    for (int i = startYear; i <= endYear; ++i) {
        years.push_back(i);
    }

    // Linear regression calculations
    int n = years.size();
    double sumX = std::accumulate(years.begin(), years.end(), 0.0);
    double sumY = std::accumulate(historicalAverages.begin(),
historicalAverages.end(), 0.0);
    double sumXY = 0.0;
    double sumX2 = 0.0;

    for (int i = 0; i < n; ++i) {
        sumXY += years[i] * historicalAverages[i];
        sumX2 += years[i] * years[i];
    }

    double slope = (n * sumXY - sumX * sumY) / (n * sumX2 - sumX *
sumX);
    double intercept = (sumY - slope * sumX) / n;

    // Predict temperature for the given prediction year
    return slope * predictionYear + intercept;
}

// Predict temperature difference between two months
double predictTemperatureDifference(
    const std::string& countryCode,
    int startMonth,
    int endMonth,
    int predictionYear) {
    // Calculate historical averages for the two months
    auto startMonthAverages =
calculateHistoricalMonthlyAverages(countryCode, startMonth);
    auto endMonthAverages =
calculateHistoricalMonthlyAverages(countryCode, endMonth);

    if (startMonthAverages.empty() || endMonthAverages.empty()) {
        std::cerr << "Error: Insufficient historical data for
prediction.\n";
        return 0.0;
    }

    // Predict temperatures for the specified year
    double startMonthPrediction =
predictMonthlyTemperature(startMonthAverages, predictionYear);
    double endMonthPrediction =
predictMonthlyTemperature(endMonthAverages, predictionYear);

    // Calculate and return the difference
    return endMonthPrediction - startMonthPrediction;
}

// Function to get predicted temperatures for the specified months

```

```

and year
std::vector<double> getPredictedTemperatures(const std::string&
countryCode, int startMonth, int endMonth, int predictionYear) {
    std::vector<double> predictedTemps;

    // Loop over the months in the specified range
    for (int month = startMonth; month <= endMonth; ++month) {
        // Calculate the historical monthly averages for the
specified month
        auto monthlyAverages =
calculateHistoricalMonthlyAverages(countryCode, month);

        // Use the linear regression model to predict the
temperature for the given month and year
        double predictedTemp =
predictMonthlyTemperature(monthlyAverages, predictionYear);

        predictedTemps.push_back(predictedTemp); // Store the
predicted temperature for this month
    }

    return predictedTemps; // Return the list of predicted
temperatures
}

```